

Topic 1: Variables

A variable is a container that stores data.

```
name = "Sowdhanya"  
age = 22  
cgpa = 8.9
```

Here,

- `name` → String
 - `age` → Integer
 - `cgpa` → Float
-

Topic 2: Data Types

Python is a **dynamically typed language**, which means **you don't explicitly declare the data type**. Python automatically determines it based on the value you assign.

```
a = 10      # int  
b = 2.5     # float  
c = "Python" # str  
d = True    # bool
```

Data Type	Example
int	10
float	5.6
str	"Hello"
bool	True

How can we check the memory used?

Python provides the `sys` module.

```
import sys
```

```
x = 10
```

```
print(sys.getsizeof(x))
```

Example output:

```
28
```

Topic 3: Input

```
name = input("Enter your name: ")  
print(name)
```

Everything from `input()` is stored as a **string**.

Example:

Input:
25

Actually stored as:
"25"

Topic 4: Type Casting

Convert one type into another.

```
age = int(input("Enter age: "))  
Without int():  
age = input()
```

```
print(age + age)
```

Input

```
10
```

Output

```
1010
```

Because it's a string.

With `int()`:

```
age = int(input())
print(age + age)
```

Output
20

Topic 5: Operators

What is an Operator?

An **operator** is a symbol that performs an operation on one or more values (called **operands**).

Example:

```
a = 10
b = 5

print(a + b)
```

Here:

- **a** and **b** are **operands**
- **+** is the **operator**

It tells Python to **add** the two numbers.

Types of Operators in Python

Python has **7 major types of operators**.

No.	Operator Type	Purpose
1	Arithmetic Operators	Mathematical calculations
2	Assignment Operators	Assign values
3	Comparison (Relational) Operators	Compare values

4	Logical Operators	Combine conditions
5	Identity Operators	Check if objects are the same
6	Membership Operators	Check whether an element exists
7	Bitwise Operators	Operate on binary bits

1. Arithmetic Operators

These are used for mathematical operations.

Operator	Meaning	Example
+	Addition	5 + 3
-	Subtraction	5 - 3
*	Multiplication	5 * 3
/	Division	5 / 3
//	Floor Division	5 // 3
%	Modulus	5 % 3
**	Exponent	5 ** 3

Addition (+)

```
a = 20
b = 10

print(a + b)
```

Output
30

Used to add numbers.

It also joins strings.

```
print("Hello" + " World")
```

```
Output  
Hello World
```

Subtraction (-)

```
print(20 - 5)
```

```
Output  
15
```

Multiplication (*)

```
print(5 * 4)
```

```
Output  
20
```

Strings can also be repeated.

```
print("Python " * 3)
```

```
Output  
Python Python Python
```

Division (/)

Returns the exact quotient as a float.

```
print(10 / 2)
```

```
Output  
5.0
```

Even if the answer is a whole number, `/` returns a **float**.

```
print(8 / 4)
```

```
Output  
2.0
```

Floor Division (//)

Returns only the integer part by removing the decimal.

```
print(10 // 3)
```

Output

```
3
```

Because

$10 / 3 = 3.333...$

Floor division removes the decimal part.

Difference between / and //

```
print(17 / 5)
```

Output

```
3.4
```

```
print(17 // 5)
```

Output

```
3
```

Modulus (%)

Returns the **remainder** after division.

```
print(17 % 5)
```

Calculation

$17 \div 5$

Quotient = 3

Remainder = 2

Output

2

Exponent (**)

Raises a number to a power.

```
print(2 ** 3)
```

Means

$2 \times 2 \times 2$

Output

8

Examples:

```
print(5 ** 2)
```

Output

25

2. Assignment Operators

Used to assign or update values.

`a = 10`

Here `=` assigns 10 to `a`.

Compound assignment operators

Operator	Example	Equivalent
<code>=</code>	<code>a = 10</code>	Assign
<code>+=</code>	<code>a += 5</code>	<code>a = a + 5</code>
<code>-=</code>	<code>a -= 5</code>	<code>a = a - 5</code>
<code>*=</code>	<code>a *= 5</code>	<code>a = a * 5</code>

/=	a /= 5	a = a / 5
//=	a //= 5	a = a // 5
%=	a %= 5	a = a % 5
**=	a **= 2	a = a ** 2

Example:

```
a = 10
a += 5
print(a)
```

Output
15

3. Comparison (Relational) Operators

They compare two values and return **True** or **False**.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
print(10 == 10)
```

Output
True

```
print(10 != 5)
```

Output

True

```
print(20 > 15)
```

Output

True

These are commonly used in `if` statements.

4. Logical Operators

Logical operators combine multiple conditions.

Operator	Meaning
and	Both conditions must be True
or	At least one condition must be True
not	Reverses the result

AND

age = 22

salary = 50000

```
print(age > 18 and salary > 30000)
```

Output

True

If one condition is False:

```
print(True and False)
```

Output

False

OR

```
print(True or False)
```

Output

```
True
```

Only one condition needs to be True.

NOT

```
print(not True)
```

Output

```
False
```

```
print(not False)
```

Output

```
True
```

5. Identity Operators

These check whether two variables refer to the **same object** in memory.

Operator	Meaning
is	Same object
is not	Different objects

Example:

```
a = [1, 2]  
b = a
```

```
print(a is b)
```

Output

```
True
```

Because `a` and `b` point to the same list.

Another example:

```
a = [1, 2]
b = [1, 2]

print(a is b)
```

Output
`False`

Even though the values are the same, they are different objects.

Interview Tip:

- `==` checks **whether values are equal**.
 - `is` checks **whether both variables refer to the same object**.
-

6. Membership Operators

Used to check whether a value exists in a sequence (like a string, list, tuple, or set).

Operator	Meaning
<code>in</code>	Present
<code>not in</code>	Not present

Example:

```
name = "Python"

print("P" in name)
```

Output
`True`

```
print("Z" in name)
```

Output
`False`

7. Bitwise Operators

These work on the **binary (0s and 1s)** representation of integers.

Operator	Meaning
&	AND
	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Example:

```
print(5 & 3)
Binary:
5 = 101
3 = 011

AND = 001
Output
1
```

For beginners and placement preparation, you can leave bitwise operators until later. Most entry-level coding interviews focus much more on arithmetic, comparison, logical, assignment, and membership operators.

Operator Precedence (Order of Execution)

When an expression contains multiple operators, Python follows a precedence order.

```
print(2 + 3 * 4)
```

Output:

14

Python first evaluates:

$3 * 4 = 12$

Then:

$2 + 12 = 14$

Use parentheses to control the order:

```
print((2 + 3) * 4)
```

Output:

20

Real Interview Example

```
age = 21
has_id = True

if age >= 18 and has_id:
    print("Eligible to vote")
else:
    print("Not eligible")
```

This example combines:

- Comparison operators ($\geq$)
- Logical operators (`and`)
- Boolean values (`True`)
- Conditional statements (`if`)

Logic Question 1

Write a program to add two numbers.

Think first

What is given?

Two numbers.

What should we do?

Add them.

Code

```
a = int(input())
b = int(input())

sum = a + b

print(sum)
```

Logic Question 2

Find the area of a rectangle.

Formula

Length × Breadth

l = int(input())

b = int(input())

```
area = l * b

print(area)
```

Logic Question 3

Swap two numbers (using a temporary variable)

```
a = 10
b = 20

temp = a
a = b
b = temp
```

```
print(a)
print(b)
```

Mini Quiz (Dry Run)

Q1

```
a = 5
b = 2
print(a // b)
```

Q2

```
print(10 % 3)
```

Q3

```
print(2 ** 3)
```

Q4

```
a = "10"
b = "20"
```

```
print(a + b)
```

Q5

```
a = int("15")
print(a + 5)
```

Reply with your answers in this format:

- 1.
- 2.
- 3.
- 4.
- 5.